
Université Euro-Méditerranéenne de Fès
École d'Ingénierie Digitale et d'Intelligence Artificielle

Rapport de Projet – Module de Théorie de la Commande

Self-Balancing Robot

Conception, Modélisation et Commande PID
par la Méthode de Ziegler-Nichols

Module :

Théorie de la Commande

Filière :

Ingénierie Robotique & Cobotique

Niveau :

1^{ère} Année

Réalisé par :

IVOMBO Glenn Silvère

Année universitaire :

2024 – 2025

Table des matières

1	Introduction	3
1.1	Contexte et motivation	3
1.2	Objectifs du projet	3
1.3	Organisation du rapport	3
2	Présentation du Système	4
2.1	Architecture générale	4
2.2	Composants matériels	4
2.2.1	Microcontrôleur – Arduino Uno	4
2.2.2	Centrale inertielle – MPU-6050	5
2.2.3	Driver moteur – L298N (ou équivalent)	5
2.2.4	Moteurs DC avec roues	5
2.2.5	Récapitulatif des composants	5
2.3	Schéma de câblage	6
3	Modélisation du Système	7
3.1	Analogie avec le pendule inversé	7
3.2	Variables d'état	7
3.3	Modèle linéarisé	7
4	Synthèse du Correcteur PID	9
4.1	Structure du correcteur PID	9
4.2	Méthode de Ziegler-Nichols	9
4.2.1	Principe	9
4.2.2	Application au robot	10
4.2.3	Implémentation discrète du PID	10
4.3	Sécurité : protection contre l'emballement	10
4.4	Gestion de la zone morte moteur	10
5	Implémentation Logicielle	11
5.1	Bibliothèques utilisées	11
5.2	Organigramme de fonctionnement	11
5.3	Code source complet	11
5.4	Analyse du code	14

5.4.1	Acquisition de l'angle	14
5.4.2	Gestion du temps	14
5.4.3	Anti-windup	14
6	Résultats et Discussion	15
6.1	Comportement observé	15
6.2	Analyse des paramètres PID	15
6.2.1	Effet du terme proportionnel ($K_p = 5,6$)	15
6.2.2	Effet du terme intégral ($K_i = 50$)	15
6.2.3	Effet du terme dérivé ($K_d = 0,18$)	16
6.3	Limites et perspectives	16
7	Conclusion	17

1. Introduction

1.1 Contexte et motivation

Le robot auto-équilibrant (ou *self-balancing robot*) est un système mécatronique instable par nature : à l'instar d'un pendule inversé, il tend spontanément à basculer sous l'effet de la gravité. Maintenir un tel système en équilibre constitue un problème classique et fondamental de la théorie de la commande, illustrant de manière concrète les notions de stabilité, de rétroaction et de correction dynamique.

Ce projet s'inscrit dans le cadre du module de **Théorie de la Commande** en première année du cycle ingénieur en Robotique et Cobotique. Il vise à concevoir, réaliser et commander un robot bipode à roues capable de maintenir son équilibre de façon autonome, en s'appuyant sur un correcteur **PID** (*Proportionnel-Intégral-Dérivé*) dont les gains ont été déterminés par la **méthode de Ziegler-Nichols**.

1.2 Objectifs du projet

- Modéliser le comportement dynamique d'un pendule inversé sur roues.
- Concevoir et implémenter un correcteur PID en temps réel sur microcontrôleur Arduino.
- Déterminer les gains optimaux via la méthode de Ziegler-Nichols.
- Valider expérimentalement la stabilité du système en boucle fermée.

1.3 Organisation du rapport

Ce rapport est structuré comme suit : le chapitre 2 présente le matériel utilisé ; le chapitre 3 établit le modèle mathématique du système ; le chapitre 4 détaille la synthèse du correcteur PID et l'application de Ziegler-Nichols ; le chapitre 5 décrit l'implémentation logicielle ; enfin, le chapitre 6 présente les résultats obtenus.

2. Présentation du Système

2.1 Architecture générale

Le robot est constitué d'une structure à deux étages imprimée en 3D (filament PLA, couleur bleue), montée sur deux roues à entraînement direct. Cette architecture compacte permet de concentrer l'électronique sur plusieurs niveaux, réduisant ainsi l'encombrement latéral et abaissant le centre de masse de la structure mécanique.

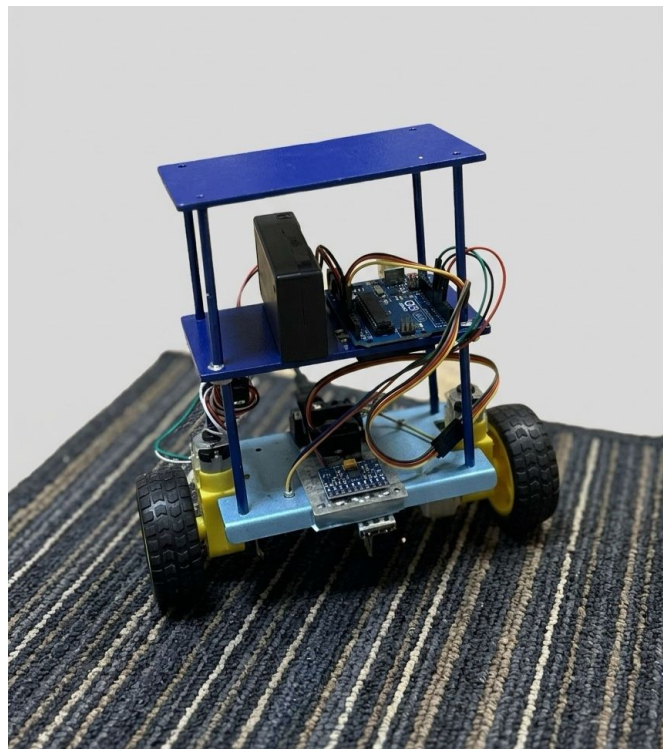


FIGURE 2.1 – Vue d'ensemble du robot self-balancing réalisé

2.2 Composants matériels

2.2.1 Microcontrôleur – Arduino Uno

L'**Arduino Uno** (microcontrôleur ATmega328P, 16 MHz) constitue l'unité centrale de traitement. Il assure :

- La lecture des données de l'IMU via le bus I²C,
- Le calcul du correcteur PID en temps réel,
- La génération des signaux PWM pour le pilotage des moteurs.

2.2.2 Centrale inertielle – MPU-6050

Le **MPU-6050** est une unité de mesure inertielle (IMU) 6 axes combinant :

- Un **accéléromètre 3 axes** ($\pm 2g$ à $\pm 16g$),
- Un **gyroscope 3 axes** (± 250 à ± 2000 °/s),
- Un processeur de mouvement numérique intégré (**DMP**).

Le DMP (*Digital Motion Processor*) est exploité via la bibliothèque `MPU6050_6Axis_MotionApps20`, permettant d’obtenir directement l’orientation sous forme de **quaternions**, puis les angles de Euler (lacet, tangage, roulis). L’angle de **tangage** (pitch) est l’unique variable d’état utilisée pour la commande.

2.2.3 Driver moteur – L298N (ou équivalent)

Le module driver moteur pilote les deux moteurs DC en courant, à partir des signaux logiques de l’Arduino :

- **ENA / ENB** (pins 10 et 5) : contrôle de la vitesse par PWM,
- **IN1, IN2 / IN3, IN4** (pins 9,8,7,6) : contrôle du sens de rotation.

2.2.4 Moteurs DC avec roues

Deux **moteurs DC à réducteur** (moteurs jaunes type TT) entraînent directement les roues. Un seuil de démarrage (*deadzone*) de 80/255 en PWM est appliqué pour compenser la zone morte mécanique des moteurs.

2.2.5 Récapitulatif des composants

Composant	Référence	Rôle
Microcontrôleur	Arduino Uno (ATmega328P)	Calcul PID, PWM
IMU	MPU-6050	Mesure de l’angle
Driver moteur	L298N (ou compatible)	Pilotage moteurs
Moteurs	Moteurs DC TT + roues	Actionneurs
Structure	PLA imprimé 3D	Châssis
Alimentation	Batterie Li-ion	Alimentation globale

TABLE 2.1 – Composants du robot self-balancing

2.3 Schéma de câblage

Arduino Pin	Signal	Destination
5	ENB (PWM)	Driver moteur – Moteur B
6	IN4	Driver moteur
7	IN3	Driver moteur
8	IN2	Driver moteur
9	IN1	Driver moteur
10	ENA (PWM)	Driver moteur – Moteur A
A4 (SDA)	I ² C Data	MPU-6050 SDA
A5 (SCL)	I ² C Clock	MPU-6050 SCL

TABLE 2.2 – Attribution des broches Arduino

3. Modélisation du Système

3.1 Analogie avec le pendule inversé

Le robot auto-équilibrant est modélisable comme un **pendule inversé sur chariot** : le corps du robot joue le rôle du pendule, les roues celui du chariot. Ce système est intrinsèquement instable : sans commande active, tout écart angulaire s'amplifie exponentiellement.

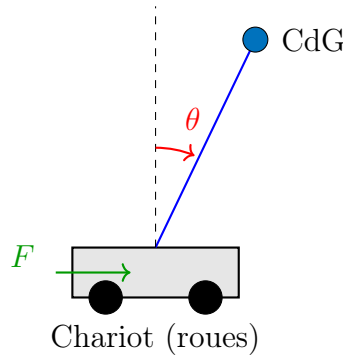


FIGURE 3.1 – Modèle pendule inversé – analogie avec le robot

3.2 Variables d'état

Le système est caractérisé par :

$$\theta : \text{angle de tangage (pitch) mesuré par le MPU-6050} \quad (3.1)$$

$$\theta_0 = 2,15 : \text{point d'équilibre (setpoint)} \quad (3.2)$$

$$e(t) = \theta(t) - \theta_0 : \text{erreur de commande} \quad (3.3)$$

Le point d'équilibre n'est pas exactement 0° en raison du déséquilibre mécanique du centre de masse du robot (distribution des composants, câbles, etc.).

3.3 Modèle linéarisé

Autour du point d'équilibre, la dynamique simplifiée du pendule inversé s'écrit :

$$(I + mL^2)\ddot{\theta} = mgL \sin(\theta) - \tau \quad (3.4)$$

Pour θ petit ($\sin \theta \approx \theta$), la linéarisation donne :

$$\ddot{\theta} \approx \frac{mgL}{I + mL^2} \theta - \frac{1}{I + mL^2} \tau \quad (3.5)$$

La fonction de transfert en boucle ouverte est du second ordre instable :

$$G(s) = \frac{\Theta(s)}{U(s)} = \frac{-b}{s^2 - a} \quad (3.6)$$

avec $a = \frac{mgL}{I+mL^2} > 0$ (terme déstabilisant) et $b = \frac{1}{I+mL^2}$.

Ce pôle réel positif ($s = +\sqrt{a}$) confirme le caractère **instable en boucle ouverte** du système, nécessitant une commande active.

4. Synthèse du Correcteur PID

4.1 Structure du correcteur PID

Un correcteur **PID** (*Proportionnel - Intégral - Dérivé*) est retenu pour sa robustesse et son adéquation aux systèmes instables du second ordre. La loi de commande est :

$$u(t) = K_p \cdot e(t) + K_i \int_0^t e(\tau) d\tau + K_d \cdot \frac{de(t)}{dt} \quad (4.1)$$

Chaque terme joue un rôle distinct :

Terme	Rôle	Effet sur la réponse	Valeur retenue
$K_p = 5,6$	Réaction proportionnelle	Réduit l'erreur statique	5,6
$K_i = 50$	Correction cumulative	Élimine l'erreur résiduelle	50
$K_d = 0,18$	Anticipation dérivée	Amortit les oscillations	0,18

TABLE 4.1 – Paramètres PID retenus et leurs rôles

4.2 Méthode de Ziegler-Nichols

4.2.1 Principe

La méthode de **Ziegler-Nichols en boucle fermée** (méthode des oscillations continues) est une procédure expérimentale de réglage dont le protocole est le suivant :

1. Annuler les actions intégrale et dérivée : $K_i = 0$, $K_d = 0$.
2. Augmenter progressivement K_p jusqu'à obtenir des **oscillations entretenues** (ni divergentes, ni amorties).
3. Relever le **gain critique** K_u et la **période critique** T_u .
4. Calculer les gains PID selon le tableau de Ziegler-Nichols :

Type de correcteur	K_p	K_i	K_d
P	$0,5 K_u$	—	—
PI	$0,45 K_u$	$\frac{K_p}{0,83 T_u}$	—
PID	$0,6 K_u$	$\frac{2K_p}{T_u}$	$\frac{K_p T_u}{8}$

TABLE 4.2 – Formules de Ziegler-Nichols pour un correcteur PID

4.2.2 Application au robot

La procédure a été appliquée expérimentalement sur le robot en conditions réelles :

1. Avec $K_i = 0$ et $K_d = 0$, le gain K_p a été augmenté progressivement.
2. Les oscillations entretenues ont été observées pour une valeur de gain critique K_u .
3. La période de ces oscillations T_u a été estimée par observation du moniteur série de l'Arduino.
4. Les formules de Ziegler-Nichols ont fourni des valeurs initiales, ensuite affinées par réglage fin expérimental.

Les valeurs finales retenues après affinage sont :

$$K_p = 5,6 \quad K_i = 50 \quad K_d = 0,18 \quad (4.2)$$

4.2.3 Implémentation discrète du PID

Le correcteur PID est implémenté en temps discret avec un pas d'échantillonnage Δt mesuré à chaque itération via `millis()`. Les équivalents discrets des termes sont :

$$\text{Intégrale : } \sum e_k \cdot \Delta t_k \quad (4.3)$$

$$\text{Dérivée : } \frac{e_k - e_{k-1}}{\Delta t_k} \quad (4.4)$$

4.3 Sécurité : protection contre l'emballement

Une condition de sécurité est intégrée : si l'angle dépasse ± 40 (chute du robot), les moteurs sont arrêtés et l'intégrateur est remis à zéro afin d'éviter le phénomène d'**intégrator windup** :

$$|\theta| > 40 \implies u = 0 \quad \text{et} \quad \sum e_k \leftarrow 0 \quad (4.5)$$

4.4 Gestion de la zone morte moteur

Les moteurs DC présentent une zone morte (friction statique) : en dessous d'un seuil PWM de 80/255, ils ne démarrent pas. La commande effective envoyée aux moteurs est donc :

$$\text{PWM} = \text{constrain}(|u| + 80, 0, 255) \quad (4.6)$$

5. Implémentation Logicielle

5.1 Bibliothèques utilisées

- `Wire.h` : communication I²C entre Arduino et MPU-6050,
- `I2Cdev.h` : couche d'abstraction I²C,
- `MPU6050_6Axis_MotionApps20.h` : interface haut niveau avec le DMP du MPU-6050.

5.2 Organigramme de fonctionnement

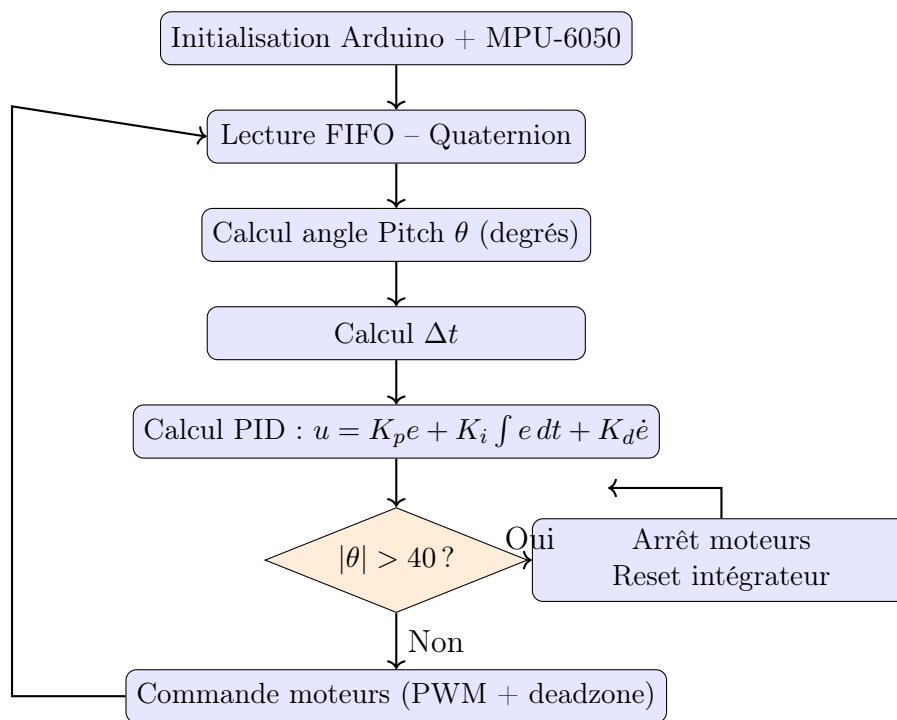


FIGURE 5.1 – Organigramme de l’algorithme de commande

5.3 Code source complet

```
1 #include "Wire.h"
2 #include "I2Cdev.h"
3 #include "MPU6050_6Axis_MotionApps20.h"
4
5 // --- CONFIGURATION PINS ---
```

```

6  const int ENA = 10; const int IN1 = 9; const int IN2 = 8;
7  const int IN3 = 7;  const int IN4 = 6; const int ENB = 5;
8
9  MPU6050 mpu;
10 bool dmpReady = false;
11 uint16_t packetSize;
12 uint8_t fifoBuffer[64];
13 Quaternion q;
14 VectorFloat gravity;
15 float ypr[3];
16
17 // --- PARAMETRES D'EQUILIBRE ---
18 float setpoint = 2.15; // Point d'équilibre en degres
19
20 // --- GAINS PID ---
21 float Kp = 5.6; // Reaction immediate (proportionnel)
22 float Kd = 0.18; // Amortissement des oscillations (derive)
23 float Ki = 50; // Elimination de l'erreur residuelle (integral)
24
25 float lastError = 0;
26 float errorSum = 0;
27 unsigned long lastTime;
28
29 void setup() {
30     Serial.begin(115200);
31     Wire.begin();
32
33     pinMode(ENA, OUTPUT); pinMode(IN1, OUTPUT); pinMode(IN2, OUTPUT);
34     pinMode(IN3, OUTPUT); pinMode(IN4, OUTPUT); pinMode(ENB, OUTPUT);
35
36     Serial.println("Initialisation du MPU...");
37     mpu.initialize();
38     if (mpu.dmpInitialize() == 0) {
39         mpu.setDMPEntered(true);
40         dmpReady = true;
41         packetSize = mpu.dmpGetFIFOPacketSize();
42         lastTime = millis();
43     }
44 }
45
46 void loop() {
47     if (!dmpReady) return;
48
49     if (mpu.dmpGetCurrentFIFOPacket(fifoBuffer)) {
50         mpu.dmpGetQuaternion(&q, fifoBuffer);

```

```

51     mpu.dmpGetGravity(&gravity, &q);
52     mpu.dmpGetYawPitchRoll(ypr, &q, &gravity);
53
54     // Angle de tangage mesure
55     float currentAngle = ypr[1] * 180 / M_PI;
56
57     // Pas de temps
58     unsigned long now = millis();
59     float dt = (now - lastTime) / 1000.0;
60     lastTime = now;
61
62     // --- CALCUL PID ---
63     float error = currentAngle - setpoint;
64     errorSum += error * dt; // Integrale
65     float dError = (error - lastError) / dt; // Derivee
66
67     float output = (error * Kp) + (errorSum * Ki) + (dError * Kd);
68     lastError = error;
69
70     // Debug serie
71     Serial.print("Angle: "); Serial.print(currentAngle);
72     Serial.print(" | Sortie: "); Serial.println(output);
73
74     // Securite chute
75     if (abs(currentAngle) > 40) {
76         stopMotors();
77         errorSum = 0; // Anti-windup
78     } else {
79         controlMotors(output);
80     }
81 }
82 }
83
84 void controlMotors(float speed) {
85     int deadzone = 80; // Seuil de demarrage moteur
86     int pwm = constrain(abs(speed) + deadzone, 0, 255);
87     bool forward = (speed > 0);
88
89     digitalWrite(IN1, forward); digitalWrite(IN2, !forward);
90     digitalWrite(IN3, forward); digitalWrite(IN4, !forward);
91
92     analogWrite(ENA, pwm);
93     analogWrite(ENB, pwm);
94 }
95

```

```
96 void stopMotors() {  
97     analogWrite(ENA, 0);  
98     analogWrite(ENB, 0);  
99 }
```

Listing 5.1 – Code Arduino – Self-Balancing Robot

5.4 Analyse du code

5.4.1 Acquisition de l'angle

Le DMP du MPU-6050 fusionne les données accéléromètre et gyroscope pour produire des quaternions unitaires. Ces quaternions sont convertis en angles d'Euler (yaw, pitch, roll). Seul l'angle de **pitch** (`ypr[1]`) est utilisé, converti de radians en degrés.

5.4.2 Gestion du temps

La fonction `millis()` retourne le temps écoulé depuis le démarrage en millisecondes. Le pas d'échantillonnage Δt est calculé à chaque itération, garantissant un calcul correct de l'intégrale et de la dérivée indépendamment de la charge CPU.

5.4.3 Anti-windup

Lorsque le robot dépasse ± 40 , l'accumulateur d'intégrale `errorSum` est remis à zéro. Sans cette mesure, l'intégrale continuerait de croître pendant la chute (*windup*), provoquant une commande excessive au redémarrage.

6. Résultats et Discussion

6.1 Comportement observé

Le robot démontre une capacité d'équilibrage stable autour du setpoint fixé à $\theta_0 = 2,15$. Les observations expérimentales sont les suivantes :

Démonstration vidéo du robot en fonctionnement

Une vidéo illustrant l'équilibrage autonome du robot est disponible à l'adresse suivante :

<https://www.youtube.com/shorts/oZE1ddENZmM>

(cliquer sur le lien dans la version numérique du document)

- **Équilibre statique** : le robot maintient sa position verticale de façon autonome lorsqu'il est placé sur une surface plane.
- **Réponse aux perturbations** : suite à une impulsion extérieure (poussée légère), le robot corrige sa trajectoire et retrouve l'équilibre en quelques dixièmes de secondes.
- **Correction du setpoint** : le point d'équilibre à 2,15 (et non 0) compense le déséquilibre mécanique dû à la répartition asymétrique des composants.
- **Zone morte** : le seuil PWM de 80/255 permet aux moteurs de démarrer sans à-coup.

6.2 Analyse des paramètres PID

6.2.1 Effet du terme proportionnel ($K_p = 5,6$)

Le terme K_p assure la réaction immédiate aux écarts angulaires. Une valeur trop faible rend le système lent et instable; une valeur trop élevée provoque des oscillations divergentes. La valeur de 5,6 offre un bon compromis réactivité/stabilité.

6.2.2 Effet du terme intégral ($K_i = 50$)

Le terme K_i , relativement élevé, compense les erreurs statiques persistantes liées aux imperfections mécaniques (centre de masse décentré, frottements asymétriques). Il maintient le robot centré sur le setpoint à long terme.

6.2.3 Effet du terme dérivé ($K_d = 0,18$)

Le terme K_d amortit les oscillations rapides. Sa faible valeur est volontaire : un K_d trop élevé amplifierait le bruit de mesure de l'IMU, dégradant la commande.

6.3 Limites et perspectives

- **Surface** : l'équilibre est optimal sur surface plane ; les surfaces inclinées ou irrégulières perturbent significativement le système.
- **Bruit de mesure** : le gyroscope introduit un bruit haute fréquence amplifié par la dérivée. Un filtre passe-bas sur K_d pourrait améliorer la stabilité.
- **Commande avancée** : une commande par retour d'état (LQR) ou un observateur de Kalman permettrait une meilleure gestion du bruit et une optimisation des performances.
- **Navigation** : l'ajout d'un contrôle de direction (lacet) permettrait de transformer le système en robot mobile auto-équilibrant.

7. Conclusion

Ce projet a permis de concevoir et de réaliser un robot auto-équilibrant fonctionnel, illustrant de façon concrète les principes fondamentaux de la théorie de la commande étudiés en cours.

Le système repose sur un correcteur **PID discret** dont les gains ont été déterminés expérimentalement par la **méthode de Ziegler-Nichols**, puis affinés par réglage fin. L'utilisation du DMP intégré au MPU-6050 assure une estimation fiable et filtrée de l'angle de tangage, clé de voûte de la commande en boucle fermée.

Les résultats expérimentaux valident la stabilité du système : le robot maintient son équilibre de façon autonome et récupère correctement les perturbations extérieures modérées. Ce projet constitue également une introduction pratique aux défis de l'implémentation temps réel sur microcontrôleur : gestion du pas d'échantillonnage variable, anti-windup, et compensation des non-linéarités mécaniques.

Les perspectives d'amélioration incluent l'adoption d'une commande optimale (LQR) ou d'un filtre de Kalman, qui feront l'objet d'études approfondies dans les années suivantes du cycle ingénieur.

Bibliographie

- [1] J.G. Ziegler and N.B. Nichols, *Optimum Settings for Automatic Controllers*, Transactions of the ASME, vol. 64, pp. 759–768, 1942.
- [2] K. Ogata, *Modern Control Engineering*, 5th edition, Prentice Hall, 2010.
- [3] Arduino, *Arduino Uno Reference Documentation*, <https://www.arduino.cc/en/Guide/ArduinoUno>, 2024.
- [4] InvenSense, *MPU-6050 Product Specification Revision 3.4*, TDK InvenSense, 2013.
- [5] J. Rowberg, *i2cdevlib – MPU6050 Library*, <https://github.com/jrowberg/i2cdevlib>, 2024.